

RepViT 轻量图像分类模型复现、优化与多 模型部署

2026 秋招 · one 团队 AI 算法组

目 录

2026 秋 one 团队 AI 算法组考核报告

2026 秋 one 团队 AI 算法组考核报告

一、任务背景与复现范围

1.1 任务背景

阅读并理解论文《RepViT: Revisiting Mobile CNN From ViT Perspective》（arXiv:2307.09283, CVPR 2024），基于官方开源项目完成 RepViT 的运行评价、迁移训练、模型优化、结果可视化、结构重参数化与 ONNX 部署。论文的核心论点是：RepViT 名称虽含 "ViT"，但**不含自注意力模块**，其性能来自轻量 ViT 的宏观架构选择与训练策略。

1.2 复现范围

范围	内容
做	官方 ImageNet-1K 权重在自建 1000 张子集上的评价（2 个基础型号 + 3 个家族型号）；M0.9 在 Oxford-IIIT Pet 37 类的迁移训练；4 组控制变量优化 + 3 组组合消融；曲线/混淆矩阵/Grad-CAM；结构重参数化数值验证；3 个 ONNX 模型的 CPU 部署与性能测试
不做	不在完整 ImageNet-1K 上从头训练；不追求论文 300/450 epoch 精度；不做第八章拓展任务（INT8 量化、知识蒸馏、剪枝、多随机种子、边缘设备部署、核心模块独立实现）

1.3 环境指纹

项	值
Python / torch / torchvision	3.14.5 / 2.14.0+cu126 / 0.29.0+cu126
timm / ONNX / ONNX Runtime	1.0.29 / 1.22.0 / 1.30.0 (CPUExecutionProvider)

GPU / CPU	RTX 4060 Laptop 8 GB (CUDA 12.6) / i7-13650HX (20 逻辑核)
内存 / 操作系统	15.8 GB / Windows 11 10.0.26200

选 ONNX Runtime 1.30.0: 1.29.0 / 1.29.1 存在 CPU FP16 Gemm 回退缺陷, 会污染基准测试。

完整快照见 `outputs/env_snapshot.json`。

1.4 三个必须区分的数字口径

同一模型在不同口径下的参数量**都是对的**, 不标口径就是错的:

口径	M0.9 参数量	说明
训练态双头 (含蒸馏头, C=1000)	5,489,328	timm 裸 create_model 默认
未融合单头 (C=1000 / C=37)	5,103,560 / 4,732,805	官方 README 的 5.1M / 本任务训练态
骨干 (不含任何分类头)	4,717,792	
融合后单头 (C=1000 / C=37)	5,067,056 / 4,696,301	结构重参数化之后

MACs 亦有 thop 与 fvcare 两种口径: thop = **847,050,816** (0.847 GMACs) , fvcare = **832,165,824** (同口径, 不是 2 倍) 。

二、RepViT 论文核心思路

2.1 为什么叫「从 ViT 视角重新审视 Mobile CNN」

轻量 ViT 在移动端同时取得更好的精度与延迟, 业界通常把功劳归给多头自注意力 (MHSA)。论文用**增量式架构改造实验**证明: 真正起作用的是轻量 ViT 的宏观架构与训练策略, 而非 MHSA。做法是从 MobileNetV3 出发逐步引入轻量 ViT 的高效设计, 每一步只改一处, 最终得到 RepViT 家族。

2.2 为什么 RepViT 仍是纯 CNN

因为它**没有任何自注意力算子**: 全局信息交互通过 1×1 卷积 (Channel Mixer) 与 depthwise 卷积 (Token Mixer) 实现, 感受野靠堆叠与下采样扩大。在 ONNX 计算图中可直接验证: 全部算术节点为 Conv / Gemm / Add / Mul / Div / Clip / Relu 等, **没有一个 MatMul、没有 Softmax 用于注意力**。

2.3 关键设计逐条

设计	内容	动机
Token Mixer / Channel Mixer 分离	每个 Block 先空间混合 (Token Mixer) 再通道混合 (Channel Mixer)，中间加残差；解耦后可对空间混合只用极轻的 depthwise	与 ViT 的 MHSA + FFN 一一对应；容量可独立调节，不像 MobileNetV2 倒残差把空间与通道耦合
RepVGGDW Token Mixer	训练态 = 3×3 depthwise 分支 + 1×1 depthwise 分支 + BN；推理态融合成单个 3×3 depthwise	训练时多分支提供更强表达能力，推理时零开销（见第十二节）
降低扩张比例 + 增加宽度	扩张比从 MobileNetV3 的 6 降到 2 左右，同时整体加宽	扩张比降低显著减少逐点卷积 MACs，省下的预算用于加宽，同等 MACs 下精度更高
Early Convolution Stem	用两组 stride=2 的 3×3 卷积代替 ViT 的 patchify	patchify 是 stride=16 的大核非重叠卷积，局部纹理建模弱；早期小幅下采样保留更多细节
更深的下采样层	下采样放在更深的层级	浅层分辨率高，过早下采样会丢失细粒度信息
SE 模块非每块都放	仅部分 Block 后加 Squeeze-Excite	SE 带来全局信息，但每次要全局池化 + 两次全连接，延迟不可忽略，只在收益最大处放
简单分类头	单个 BN + Linear (NormLinear)	复杂头（多头/多层 MLP）是实打实的推理延迟，小模型上精度收益有限
depthwise + pointwise 分工	depthwise 做空间局部混合（每通道独立，参数量 $\approx k^2C$ ）； pointwise 做跨通道混合（ 1×1 ， $\approx C^2$ ）	把标准卷积的 C^2k^2 降到 $C^2 + k^2C$ ，是移动端卷积的标准拆解

2.4 五个型号的差别

同一套 Block 设计在不同宽度/深度上的缩放，差别集中在每个 stage 的通道数与 Block 数。实测 M0.9 的 4 个 stage Block 数为 **[2, 2, 14, 2]**（共 20 个）。

三、RepViT-M0.9 结构

下图由 tools/draw_arch.py 用 forward hook **现测**真实张量形状生成（非论文插图、非手画）。

```

输入  224 × 224 × 3 (RGB)
|
Stem  stem.conv1  3×3 stride=2 → 24ch  112×112    ← Early Convolution Stem 前半
      stem.conv2  3×3 stride=2 → 48ch   56×56      ← Early Convolution Stem 后半
|
Stage 0  stages.0   2× RepViT Block  48ch  56×56
Stage 1  stages.1   2× RepViT Block  96ch  28×28
Stage 2  stages.2  14× RepViT Block 192ch  14×14
Stage 3  stages.3   2× RepViT Block 384ch   7×7
|
GAP  Global Average Pooling → 384 维
|
Head  RepViTClassifier → NormLinear(BatchNorm1d + Linear) → 37 维
  
```

- 分辨率 $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$ （总下采样 $32\times$ ）；通道 $3 \rightarrow 24 \rightarrow 48 \rightarrow 96 \rightarrow 192 \rightarrow 384$ 。
- **Stage 2 独占 14 个 Block**（共 20 个），与论文「第三阶段布置更多 Block」一致。
- 每个 Block = Token Mixer（RepVGGDW： $3\times 3 + 1\times 1$ depthwise 双分支）+ Channel Mixer（ 1×1 升维 $\rightarrow$ GELU $\rightarrow 1\times 1$ 降维）+ 残差；部分 Block 带 SE。
- **训练态 vs 推理态**：训练态 Block 内含多分支与 BN；推理态融合后 Token Mixer 变成**单个 3×3 depthwise 卷积**。产物：outputs/architecture/repvit_m0_9_arch.png。

四、官方预训练模型评价

4.1 评价口径

项	值
验证子集	1000 张 （1000 类各 1 张，分层抽样，种子 20260912）， datasets/lists/imagenet_val_subset.txt
预处理	Resize(256, bicubic) $\rightarrow$ CenterCrop(224) $\rightarrow$ Normalize(ImageNet)，即 crop_pct = 0.875（官方

	口径)
权重 / 实现	官方 Releases v1.0 的 *_distill_300e.pth; vendered 官方实现 + replace_batchnorm 融合态
设备	RTX 4060 Laptop, FP32, batch=64

声明：该子集是**自建**分层抽样清单，**不是考核方下发的指定子集**；以下结果只代表「该子集上的实际运行结果」，不代表完整 ImageNet-1K 验证集。清单哈希见上表路径。

4.2 实测结果

型号	参数量 (训练态双头)	MACs (thop)	Top-1 (%)	Top-5 (%)	文件大小
RepViT-M0.9	5,489,328	0.847 G	78.20	93.70	21.38 MB
RepViT-M1.0	7,302,796	1.143 G	79.90	94.20	28.30 MB

预处理口径对照（同批图片，只改 crop_pct）：M0.9 = 78.20/93.70 (0.875) vs 78.60/93.00 (0.95)；M1.0 = 79.90/94.20 vs 80.00/93.90。两套口径差 0.1~0.4 个点，**不可混用**，两套都记录在 outputs/pretrained_eval/<model>/metrics.json 的 crop_pct_sweep 字段。

4.3 与论文/官方公布值的三方对照

	论文 / 官方公布	本机实测 (自建子集)	差值
M0.9 Top-1	78.7 %	78.20 %	-0.50
M1.0 Top-1	80.0 %	79.90 %	-0.10

差值合理：子集只有 1000 张（每类 1 张），单张图片翻转就值 0.1 个百分点；且官方公布值是完整 50,000 张验证集的结果。

4.4 延迟 (PyTorch 侧)

型号	融合后 mean (ms)	P50	P95	未融合 mean (ms)
M0.9	7.97	7.79	9.22	15.51
M1.0	8.53	8.06	12.46	22.63

口径：PyTorch 2.14.0+cu126, RTX 4060 Laptop, FP32, batch=1, 224×224, threads=4,

预热 10 次 + 正式 50 次。融合后明显低于未融合，**这是结构重参数化在 PyTorch 侧的收益**

(M0.9 降 48.6%、M1.0 降 62.3%)。官方公布的 iPhone 12 延迟 (0.9 / 1.0 ms) 与本机

不可直接比较——设备、框架、精度、协议全不同。

4.5 Top-5 样例与正误案例

每型号落盘 ≥6 张图片的 Top-5 类别与置信度，并配正误案例图 (outputs/pretrained_eval/<model>/)。 **正**

确案例：一张 beagle 图以 96.8% 置信度判对，Top-5 其余四项 (basset、bloodhound、bluetick、black-and-tan coonhound) 全是猎犬类——说明模型的「备选项」在语义上也是合理的。 **错误案例**：细长体型小型犬 (如 dachshund) 被判成其他小型猎犬；主色调相近但体型不同的品种混判。

4.6 较大模型是否在当前设备上获得了合理收益

M1.0 相比 M0.9：参数量 +33.0%、MACs +34.9%，Top-1 只提升 **+1.70 个点** (78.20 → 79.90)，而融合后延迟只增加 7.0%。 **结论：在当前 CPU 部署目标下 M1.0 的收益不划算**——CPU 实时性为约束时应选 M0.9；收益要用「精度增量 / 延迟增量」衡量，而不是单看精度。

五、多型号规模和性能比较

口径：同一子集 (1000 张)、crop_pct=0.95 + bicubic、ONNX Runtime CPUExecutionProvider、threads=4、batch=1、FP32、预热 10 + 正式 50。延迟取自 outputs/benchmarks/*_benchmark.json。

型号	参数量 (M) 融合后单 头	MACs (G) thop 未融 合	ONNX 文件 (MB)	本机 Top-1 (%)	官方公布	P50 (ms) ORT CPU	P95 (ms)
RepViT-M0.9	5.067	0.847	20.36	78.20	78.7	7.37	8.00
RepViT-M1.0	6.810	1.143	27.33	79.90	80.0	9.15	9.76
RepViT-M1.1	8.244	1.377	33.06	79.90	80.7	10.23	11.24
RepViT-M1.5	14.050	2.340	56.35	82.80	82.3	17.65	18.93
RepViT-M2.3	22.927	4.626	91.90	83.10	83.3	32.69	33.32

边际收益（每多花 1 ms 换来的 Top-1 增量）：M0.9 → M1.0 = **+0.96**（划算）；M1.0 → M1.1 = **0.00**（完全不划算）；M1.1 → M1.5 = +0.39；M1.5 → M2.3 = +0.02。

帕累托与推荐：P50 ≤ 15 ms 预算下推荐 repvit_m0_9_in1k（得分 0.9842，P50 7.37 ms、Top-1 78.20%）；预算放宽到 10 ms 且优先精度时 repvit_m1_0_in1k 是帕累托最优（Top-1 79.90%、P50 9.15 ms）。M1.1 零精度增益却多 1.09 ms，属被支配点；M2.3 付出 4.4 倍延迟只换来 0.3 个点。

数据来源：outputs/pretrained_eval/summary.csv（参数量、MACs）、outputs/benchmarks/summary.csv（ONNX P50/P95）、outputs/advanced/*。

注意不要混用：outputs/benchmarks/family_summary.csv 的 params_M 是训练态双头口径（M0.9 = 5.489M）、延迟是 PyTorch CPU 计时（M0.9 = 36.1 ms），与上表口径不同。

六、数据集与数据划分

6.1 两个数据集，两套标签，绝不混用

	Oxford-IIIT Pet	ImageNet-1K 验证子集
用途 / 类别数	迁移训练 train/val/test； 37 类	官方预训练模型评价； 1000 类
标签文件	labels/pet_classes.txt	labels/imagenet_classes.txt
规模	7390 张有标注图片	1000 张（自建分层子集）

deploy/model_registry.py 用 labels_for(key) 按模型强制选择标签文件，并有硬断言（行数 ≠ num_classes 立即失败）——37 类与 1000 类混用会让 Top-5 类别名整体错乱。

6.2 官方数据事实基线（实测）

事实	实测值
images/ 下 jpg / list.txt 数据行	7390 / 7349
annotations/trainval.txt / test.txt	3680 行 / 3669 行
类别编号 / 猫狗类别数	官方 CLASS-ID 是 1-based （0-based 标签 = CLASS-ID - 1）； 12 / 25

官方**不存在** train.txt / val.txt，只有 trainval.txt 与 test.txt，train/val 必须

自行分层切分；且官方 CLASS-ID 顺序**猫狗交错**（idx 0 Abyssinian=猫，idx 1 American

Bulldog=狗 ...），任何「前 12 类都是猫」的假设都会让物种分析整体错位。

6.3 划分方案

trainval.txt (3680) → 分层切分（类内 sorted 后 shuffle，每类固定 20 张进 val）

└─ pet_train.txt 2940 行（每类 73~80 张） └─ pet_val.txt 740 行（每类恰好 20 张）

test.txt (3669) → pet_test.txt 3669 行（每类 88~100 张）

冻结信息：seed=42、val_per_class=20、行格式 <image_id>\t<class_idx_0based>； 实验中途未变更过划分。

6.4 数据泄漏检查（三层 + 内容级）

检查项	结果
文件名交集 train∩val / train∩test / val∩test	0 / 0 / 0
MD5 交集（防同一张图换名后跨划分）	0 / 0 / 0
孤儿图误入 / 划分内部重复图	0 / 0

命令：python datasets/audit_leakage.py → outputs/metrics/leakage_check.json。 **test 集只被评价过一次**

（outputs/metrics/baseline_test.json 的 eval_count = 1）， 全程未用于模型选择、调参或早停；选模唯一依据是验证集 val_macro_f1。

七、Baseline 迁移训练

7.1 训练配置

项	值
模型 / 初始化	RepViT-M0.9, impl = timm, distillation = False (单头)；ImageNet-1K 预训练权重（与官方 .pth 逐位相同，见 7.3）
数据 / 输入	Oxford-IIIT Pet 37 类；224×224；train batch 64 (drop_last=True → 45 step/epoch)，eval 128
Loss / 优化器	Cross Entropy + Label Smoothing 0.1；AdamW, lr 1e-3, backbone 倍率 0.1, weight decay 0.05 (BN/bias 关闭)

调度 / 精度	Cosine, warmup 3 epoch (起始因子 0.01) , min_lr 1e-5; bfloat16 autocast (仅 CUDA)
epochs / seed	40 (满预算, 早停等价于关闭) ; 42 (固定 random/numpy/torch/cuda/PYTHONHASHSEED)

训练预算：全部实验臂统一 40 epoch, early_stop_patience = 999 (保留机制、默认不触发)。

原因是预算不等价会直接毁掉控制变量对比——早期用默认 patience=8 跑 opt_mix 时它在 epoch 24 早停 (25 epoch) , 与 40 epoch 的 baseline 差 37.5%。判别: python tools/same_budget.py。

7.2 结果

指标	验证集 (best epoch)	测试集 (唯一一次)
Top-1 Accuracy	—	92.34 %
Top-5 Accuracy	—	99.26 %
Macro-F1	0.9462	92.22 %
样本数	740	3669

骨干确实被训练 (不是只训分类头) : tools/check_backbone_updated.py 实测 changed_tensors = 706, 其中骨干张量 **699** 个变化。(题目第 27 页: 只训分类头且未更新 任何骨干阶段, 基础训练部分最高不超过 60%。) **换头方式**: 直接以 timm.create_model('repvit_m0_9', pretrained=True, num_classes=37, distillation=False) 建网, 不使用 reset_classifier (会丢 head.head.bn 统计量, 且 distillation=False 时 head_dist 被删除、断言必失败); 加载官方权重时按前缀剔除与类别数 绑定的 Linear, **保留**分类头 BN 统计量。**权重加载**: missing = 2 (仅换头后的分类层)、unexpected = 0, 报告见 outputs/metrics/weight_load_report.json。

7.3 与官方代码的差异 (题目明确要求说明)

项	官方仓库	本任务
Python / timm	3.8 / 0.5.4	3.14.5 / 1.0.29
权重载入	model/repvit.py + utils.replace_batchnorm	同等价: same
训练循环	main.py + engine.py + utils.py	自撰 tools/train.py

关键实测结论：官方 .pth (713 键, features.N.* / classifier.classifier.*) 与 timm 实现 (389 键, stages.M.* / head.head.*) **键名交集为 0**，跨体系加载必然 unexpected = 713；但两条路线**权重数值逐位相同**（同一批固定输入上，池化特征 / 主头 logits / 蒸馏平均 logits 三个口径 $\max|\Delta| = 0.000e+00$ ）。因此不需要手写 713→389 的键转换器，迁移训练可安全地从 timm 侧权重起步。证据：

outputs/metrics/weight_load_report.json 的 equivalence_official_vs_timm。

八、优化方法与实验假设

8.1 Baseline 存在的问题（优化动因）

训练集仅 2940 张而模型有 4.73M 参数（约 1600 倍于样本数）→ 过拟合风险；Label Smoothing 0.1 只能部分缓解决策面过尖的问题；分类头随机初始化需要 $1e-3$ 量级学习率，主干若同用会破坏预训练特征；增强只有 RandomResizedCrop + 水平翻转 + 轻度 ColorJitter，缺少旋转/剪切/色调类变换。

8.2 四个方案的实验假设

方案	改动	假设	预期方向
B 混合增强	Mixup($\alpha=0.2$) + CutMix($\alpha=1.0$), mixup_prob=1.0	软标签平滑决策面，等价于训练集扩容，train-val gap 收窄	需要更多 epoch 收敛
C 加强增强	RandAugment(n=2, m=9) + RRC scale 下限 $0.35 \rightarrow 0.25$ + ColorJitter 加强	覆盖旋转/剪切/色调变换，压缩对局部纹理的记忆	有过增强风险
D 差异化学习率	backbone_lr_scale 0.1 $\rightarrow$ 0.05（主干有效 lr $1e-4 \rightarrow 5e-5$ ）	新头继续用 $1e-3$ 快速收敛，主干更牢锚定在预训练解附近	前期 val 曲线更稳
A 组合式	B + C + D 全开（12 个差异键）	三者机制不冲突，呈现「前段更稳 + 末段更高」	正则叠加会延长收敛

8.3 控制变量的工程化定义

控制变量不靠自觉而是**可执行校验**：tools/diff_config.py 做叶子级配置比对，要求实际差异键与配置里声明的 _expected_diff_ 逐项一致，否则 DIFF PASS 不成立。7 份配置全部通过（差异键数：opt_mix 3、opt_randaug 8、opt_disc 1、opt_combo 12、opt_abl_a 8、opt_abl_b 1、opt_abl_ab 9）。保持不变的内容：数据划分、预训练权重、随机种子、训练轮数（40）与每轮迭代数、评价方式、best 模型选择标准（val_macro_f1）。

九、定量结果

全部数字由 tools/compare_runs.py 从 outputs/logs/*_metrics.csv 与 outputs/metrics/*_test.json 汇总，见 outputs/logs/opt_compare_summary.csv。

全部 8 组实验（每组 40 epoch 满预算，seed=42，同一划分、同一评价方式）：

实验	差异键	val Top-1 (best) %	val Macro-F1 (best)	test Top-1 %	test Top-5 %	test Macro-F1 %
Baseline	—	94.60	0.9462	92.34	99.26	92.22
opt_mix (B 混合)	3	94.86	0.9489	92.42	99.43	92.27
opt_randaug (C 增强)	8	94.46	0.9450	92.18	99.35	92.06
opt_disc (D 差分 lr)	1	94.19	0.9421	92.20	99.48	92.11
opt_combo (A 组合)	12	94.86	0.9489	92.12	99.59	92.01
opt_abl_a (= C)	8	95.00	0.9502	92.48	99.40	92.36
opt_abl_b (= D)	1	94.32	0.9433	92.53	99.32	92.44
opt_abl_ab (= C+D)	9	94.87	0.9488	91.93	99.48	91.84

必须诚实说明：8 组实验的 test Top-1 落在 **91.93 ~ 92.53** 的 0.6 个百分点窄带内，名次在不同指标间还会互换（如 opt_ab1_b 的 test Top-1 最高、val Macro-F1 最低）。**这不是「提升」，而是噪声**，判据三条：① 3669 张测试集上 Top-1 ≈ 0.92 时二项分布 95% 置信区间半宽约 **± 0.9 个点**，观测到的 0.6 个点跨度完全落在区间内；② 同一配置（baseline）重跑一次的 test Top-1 从 92.26 变到 92.34 (**± 0.08 个点**)、val Macro-F1 从 0.9502 变到 0.9462 (**± 0.40 个点**)，单次运行波动本身就有这个量级；③ 优化方法与结果之间没有单调关系（增强最弱的 D 排名靠前，三合一的 A 反而垫底）。**结论：在本任务预算（40 epoch、2940 张训练图）下，四项优化方法都没有带来超出随机波动的真实增益——这本身是有价值的负面结论。**

9.1 组合消融：方法之间是否存在叠加或冲突

指标	ΔA	ΔB	$\Delta A+B$	交互项 $I = \Delta AB - (\Delta A + \Delta B)$	判定
val Top-1	+0.000	-0.676	-0.135	+0.541	超加性（协同）
val Macro-F1	-0.000	-0.007	-0.001	+0.006	可加
val Top-5	+0.135	-0.405	+0.000	+0.270	可加

读法：单独用 B（差异化学习率）会让 val Top-1 掉 0.68 个点，A+B 一起用只掉 0.14 个点，**A 抵消了 B 的大部分伤害**。机制上说得通：B 压低主干学习率，A（RandAugment）提高数据难度，两者对主干有效更新量方向相反，叠加后回到中间值。数据来源：outputs/advanced/ablation_summary.{csv,json}、outputs/metrics/ablation.csv。

9.2 训练预算等价性

python tools/same_budget.py 实测 8 组实验的 (epochs, steps_per_epoch) 均为 **(40, 45)**，total_iters_equal = True。归因结论（假设/预期/观测/判定/解释）见 outputs/metrics/attribution.json。

十、曲线与混淆矩阵分析

10.1 五条曲线

outputs/curves/opt_compare.png 把 Baseline 与优化模型画在**同一张图、同一坐标范围**内：train loss / val loss / val Top-1 / val Macro-F1 / learning rate 五个子图。优化模型训练损失更高是**正常现象**（Mixup/CutMix 使标签变软，损失不可直接比）；学习率在 warmup 3 epoch 后 cosine 衰减到 $1e-5$ 。

是否收敛：val Top-1 在 epoch 5 前快速上升（epoch 4 已到 91.62%）、epoch 15 后进入平台期；最后 8 轮 val Top-1 的极差为 **0.95 个百分点**（baseline；组合臂 0.54）→ 已收敛。**是否过拟合：**baseline 末轮 train_loss = 0.736、val_loss = 0.352，验证损失低于训练损失——这不是「没学过拟合」，而是训练侧有 Label Smoothing 0.1 与随机增强（训练损失被正则项抬高）。判断过拟合要看 val_loss 是否持续上升：实测它从 3.65 降到 0.35，中途有 18 次单步回升（最大 +0.022），末 8 轮稳定在 0.34~0.35 区间、没有持续上升，**未见拟合形态**。**学习率与指标的关系：**warmup 期（前 3 epoch）指标上升最快，cosine 中段（epoch 10~30）是精度主要增长区间，末段学习率降到 1e-5 时指标趋于平台。**优化方法对收敛速度的影响：**按选模指标 val_macro_f1 的最优 epoch，baseline 是 **epoch 34**（最优 0.946166），三个优化臂都更早——A（RandAugment）**31**、B（差分 lr）**32**、A+B **31**；以「val_macro_f1 首次达到自身最优值 -0.005（0.5 个百分点）以内」为进入平台的判据，baseline 为 **epoch 15**，A 与 A+B 均为 **epoch 12**、B 为 **epoch 15**。这与 §8.2 对 A 的预期「正则叠加会延长收敛」**不一致**：实测组合臂反而更早进入平台。范围限定：只覆盖本机这一次训练、单种子（seed=42）、40 epoch 满预算；且 §9 已说明四组 test Top-1 的差异落在噪声内，「收敛更早」不等于「更好」。

10.2 归一化混淆矩阵

outputs/confusion_matrix/baseline_cm.png（行归一化 row_sum = 1.0，argmax 落对角线的比例 **37/37 = 100%**，对角元均值 0.9219）。

10.3 哪些品种容易混淆

outputs/confusion_matrix/baseline_per_class.csv 按 F1 升序排列，最难的三类：

类别	support	precision	recall	F1
Staffordshire Bull Terrier	89	0.767	0.629	0.691
American Pit Bull Terrier	100	0.765	0.650	0.703
Ragdoll	100	0.762	0.770	0.766

规律：混淆集中在**同物种、外形高度相似**的品种对（斗牛梗类之间、长毛猫之间），这是任务本身的难度，而非模型缺陷。

10.4 猫狗块分析（关键结论）

指标	值
----	---

猫召回 / 狗召回	88.17 % / 94.33 %
跨物种误判占比 / 种内品种错误	3.56 % (10 / 281 个错误) / 271 个

结论：模型几乎不混淆猫与狗（3.56%），全部错误的 96.4% 是「同物种内分不清品种」。失败归因应定位到细粒度品种差异，而不是「模型没学会猫狗」。落盘：

outputs/confusion_matrix/baseline_cat_dog_block.json。

10.5 错误来自主体外观、姿态、遮挡还是背景

结合 Grad-CAM 与失败案例配图，三种主要成因：① **主体外观相似**（占多数，斗牛梗类、长毛猫类的品种间差异本身就小）；② **姿态/视角极端**（侧躺、只露头部、大幅旋转时判别性部位不可见）；③ **背景干扰**（主体占比小的图片里 Grad-CAM 注意力分散到背景纹理上）。

十一、Grad-CAM 与失败案例

11.1 Grad-CAM 实现

tools/gradcam.py 为**手写实现**（不依赖 pytorch-grad-cam），依据 Selvaraju et al., *Grad-CAM*, ICCV 2017 的公式 (1)(2)：先对目标类别 logits 反传得到特征图梯度，对梯度做全局平均得到通道权重，加权求和后过 ReLU，再双线性插值回输入尺寸，最后逐样本 min-max 归一化。

挂载层两套实现、两个粒度：A（默认）= stages[-1].blocks[-1]，形状 (B,C,7,7)，残差相加**后**的激活；B（细粒度）= 同 Block 的 channel_mixer.conv2，残差相加**前**的最后一个 1×1 卷积输出。**A 与 B 不是同一张张量、数值不同，两者不等价**；对比图见 outputs/gradcam/gradcam_layer_compare_baseline.png。**绝不能挂 model.head / model.classifier**：它们输出 2D，反传会抛 ValueError: Invalid grads shape。

11.2 结果

产物集中在 outputs/gradcam/：正确案例 gradcam_correct_baseline.png 及 6 张单图 *_correct.png，错误案例 gradcam_wrong_baseline.png 及 6 张单图 *_wrong.png，挂载层对比图 1 张，元信息 outputs/metrics/gradcam_meta.json。

11.3 Grad-CAM 是否关注到合理区域

正确案例：热力图集中在动物的头部与躯干轮廓，背景区域响应接近 0，符合「依据主体外观判别」的预期。

错误案例：典型高置信度错误（如 american_pit_bull_terrier 判成 staffordshire_bull_terrier）中，热力图**同时点亮了两类共有的特征区**（宽厚胸部与方正头部），说明模型抓到的区域本身是对的，**但该区域的判别力不足以区分这两个品种**——是任务难度而非定位错误。

11.4 模型是否出现依赖背景的现象

部分样本（约 6.2%）的热力图在背景上有明显响应，尤其是主体占比小、背景有强纹理（草地、 花纹地毯）的图片，属**数据集偏置**的典型表现。

11.5 实际图片与数据集图片的分布差异

统计量	Oxford-IIIT Pet	外部实拍图	差异
短边（中位数） / 亮度均值	339.5 px / 0.455	375.0 px / 0.503	+10.5% / 外部更亮
亮度标准差 / 边缘密度	0.218 / 0.132	0.256 / 0.148	外部光照变化与背景复杂度更大
背景复杂度比	0.478	0.593	外部背景纹理强 24%

外部实拍 8 张（beagle 90.32%、pug 43.86%、boxer 64.28%、chihuahua 67.98%、samoyed 83.50%、newfoundland 77.14%、great_pyrenees 86.95%、pomeranian 36.91%）全部判到对应品种。**8/8 判对只是小样本观察，不能证明跨域泛化或没有过拟合**；本节的观察只覆盖这 8 张已测跨集合 图片，不构成对全部外部图片的性能声明。

来源声明：external/ 图片取自 ImageNet-1K 验证集中与 Pet 同品种的样本（跨集合真实照片）。

执行时 wikimedia 系站点不可达，故用跨数据集真实照片替代，逐张登记在

external/images_manifest.csv，**本报告不声称其为原创实拍。**

11.6 置信度高是否一定代表预测可靠

不一定——实测存在置信度 > 0.9 的错误预测。原因是交叉熵在 Label Smoothing 0.1 下 并不强惩罚过度自信，且模型在训练分布外的样本上会产生「平滑但错误」的 softmax 分布。校准分析（ECE）显示置信度系统性偏高，温度缩放可显著降低 ECE。

十二、结构重参数化

12.1 训练态 RepViT Block 的分支

每个 Block 的 Token Mixer（RepVGGDW）在训练态包含：**3×3 depthwise 卷积分支（含 BN）、1×1 depthwise 卷积分支（含 BN）、两条分支输出逐元素相加**（ReLU 在 RepVGGDW 内部、相加之前）。

Channel Mixer 是标准的 1×1 → GELU → 1×1 序列，不含可融合的并行分支；Block 整体 另有一条**残差连接**。

12.2 卷积与 BatchNorm 的等价融合推导

对「Conv → BN」这一对，推理时 $\text{BN}(y) = \gamma \cdot (y - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$ 、 $y = W * x + b$ ，代入展开后可写成**单个卷积**：

$$W' = W \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} \quad (\text{逐输出通道缩放})$$

$$b' = (b - \mu) \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} + \beta \quad (\text{新的偏置})$$

即 BN 的仿射变换可被完全吸收进卷积的权重与偏置，**推理时不需要单独的 BN 算子**。**多分支融合**：把 1×1 卷积核**零填充**成 3×3，Identity 分支等价于「中心为 1、其余为 0」的 3×3 卷积核，于是各分支变成同一形状的卷积核直接相加： $W_{\text{fused}} = W_{3 \times 3} + \text{pad}(W_{1 \times 1}) + \text{pad}(W_{\text{identity}})$ ， b_{fused} 同理为各分支偏置之和。

12.3 为什么转换前后结果应基本一致

上述变换是**代数恒等**的：融合前是「两个卷积 + 两个 BN + 一个 Add」，融合后是「一个卷积」，在实数域上对同一输入产生**相同的输出**——这是代数推导，**不是位级实测结论**；浮点实现下的实际差异见 12.4 的实测值。

为什么必须在 eval() 之后融合：训练态 BN 使用**当前 batch** 的均值/方差，而融合时吸收的是 `running_mean / running_var`（训练期间累积的滑动统计量）。若在训练态融合，等价性假设（BN 用固定统计量）不成立，融合后结果会与预期不符。

12.4 实测数值对照

```
# 口径：Pet-37 baseline (timm 单头, distillation=False) , 32 个固定随机输入
# (torch.randn, seed=20240912) , batch=8, 224×224, CPU FP32, eval 后深拷贝再 fuse
python tools/reparam_verify.py --model repvit_m0_9_pet37 --weights checkpoints/baseline_best.pt \
    --num-samples 32 --batch-size 8 --seed 20240912 --out-dir outputs/reparam
```

指标	Pet-37 模型	M0.9 官方 C=1000
max_abs_err	7.092953e-06 (展示 7.093e-06)	— (旧报告权重键不匹配, 不作结论)
mean_abs_err	2.030727e-06 (展示 2.031e-06)	—
top1_identical / Top-5 最小重合	True (32/32) / 5 / 5	N/A (权重键不匹配)
BN 模块数	107 → 0	N/A
参数量 / ONNX 节点	4,732,805 → 4,696,301 (净减 36,504) / BatchNormalization 0 、Conv 103 (未融合 126, 少 23)	N/A

这里是**重参数化实验**：Pet-37 baseline 的 32 个固定随机输入（torch.randn，**不是真实测试图片**），比较训练态与融合态 PyTorch logits。落盘产物

outputs/reparam/repvit_m0_9_pet37_reparam_report.json。它与下一节的 PyTorch↔ONNX 一致性

实验不是同一批输入、也不是同一条代码路径，**两处数值不可并列成一句结论，也不能互相替代**。

重参数化减少的是**推理态**参数量：从「未融合双头 C=1000」5,489,328 降到「融合后单头 C=1000」5,067,056，净减 422,272 = 蒸馏头 385,768 + BN 折进卷积消失的 36,504。

12.5 重参数化对 ONNX 算子图的影响

指标	训练态图	推理态图	变化
节点总数 / 文件大小	480 / 较大	387 / 较小	-93
BatchNormalization	24	0	-24
Conv	126	103	-23

BatchNormalization 已归零；仍存在的算子是融合后不可避免的逐元素运算（Add / Mul / Div / Clip / Relu，来自 GELU、残差相加与逐样本归一化），**不是冗余**；少量 Identity 节点由导出器生成，对延迟无可测量影响。落盘：outputs/reparam/*_onnx_nodes.json。

12.6 单个 Block 的融合过程（以 stages.0.blocks.0 为例）

3×3 depthwise conv 与 1×1 depthwise 分支各有一组 weight 与 BN 参数（形状 (48,1,3,3) / (48,1,1,1)）；对每条分支计算 $W' = W \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)}$ 、 $b' = (b - \mu) \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} + \beta$ ，把 1×1 的核零填充到 (48,1,3,3) 后与 3×3 的核逐元素相加，最后用单个 nn.Conv2d(48, 48, 3, padding=1, groups=48) 替换整个 RepVGGDW——token_mixer 从「Sequential(两个 ConvNorm) + Add」变成一个裸 Conv2d。导出脚本：tools/reparam_deep.py。

十三、ONNX 多模型部署

13.1 三个交付模型

registry key	文件	大小	图内节点	类别数 / 标签文件
repvit_m0_9_in1k	onnx/repvit_m0_9_in1k.onnx	20.36 MB	BN=0, Conv=103, Gemm=1	1000 / labels/imagenet_classes.txt
repvit_m1_0_in1k	onnx/repvit_m1_0_in1k.onnx	27.33 MB	BN=0, Conv=103, Gemm=1	1000 / labels/imagenet_classes.txt
repvit_m0_9_pet37	onnx/repvit_m0_9_pet37.onnx	18.89 MB	BN=0,	37 / labels/pet_classes.txt

			Conv=103, Gemm=1	
--	--	--	---------------------	--

ONNX 文件名一律由 `deploy/model_registry.py` 的 `onnx_path(key)` 决定，脚本/文档/验收命令不得硬写文件名；官方 ImageNet 模型与自训练 37 类模型必须各用正确标签文件（`labels_for(key)` 强制并有硬断言）。

13.2 独立实现的部分

题目要求「独立实现图片预处理 / 独立实现 ONNX 推理 / 独立实现 Softmax 和 Top-K 后处理」。

`deploy/infer_onnx.py` 用 **PIL 手写** Resize + CenterCrop + ToTensor + Normalize，不 import torchvision。

为什么必须独立：若部署侧与训练侧共用同一份 transform，预处理 bug 会在两侧同时出现、一致率误差恒为 0，反而掩盖问题；只有两份实现互相独立，报出的一致性误差 才是有意义的证据。

13.3 PyTorch 与 ONNX 一致性

口径：Pet-37 baseline（融合态）与 `onnx/repvit_m0_9_pet37.onnx`；按 `datasets/lists/pet_test.txt` 顺序取前 12 张**真实图片**（ $n=12$ ），每张独立预处理一次后把**同一张量**送入两端（隔离预处理变量，只测模型/后端差异）；CPU FP32、batch=1、224×224。落盘 `outputs/metrics/consistency_repvit_m0_9_pet37.json`，复跑副本 `outputs/verification/consistency_repvit_m0_9_pet37_n12.json`。

关于 5.25e-06 旧引用：README 早期出现的 $5.25e-06 / 1.41e-06$ （以及旧 PPT_CONTENT.md 的 $5.245e-06 / 1.414e-06$ ）在仓库的任何提交里都没有配套产物：旧 README 的复跑命令写的是 `--limit 8`，但同一提交（81693dd）里落盘的 JSON 已经是 $n=12$ 的 $6.199e-06$ ，因此**既不能证明它来自 $n=8$ ，也不能当作 $n=12$ 的结果**；用当前权重跑 `--limit 8` 得到 $\max=6.198883056640625e-06$ 、 $\text{mean}=1.4658262017519519e-06$ ，同样无法复现旧值。旧值只作为修订记录保留，不再作为实验结论。

指标	repvit_m0_9_pet37
最大 / 平均 logits 绝对误差	6.199e-06 （落盘 $6.198883056640625e-06$ ） / $1.501e-06$ （落盘 $1.5006899711048998e-06$ ）
Top-1 类别 / Top-5 集合一致率	均 100.00%
固定测试集 Top-1 一致率 / 阈值	1.000 ($\geq 0.99 \checkmark$) / $1e-3$ ($6.199e-06 < 1e-3 \checkmark$)

复跑（写入验证目录，不覆盖正式产物）：

```
python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 \
  --images datasets/lists/pet_test.txt --limit 12 \
```

```
--out outputs/verification/consistency_repvit_m0_9_pet37_n12.json
```

这 12 张图上 Top-1 与 Top-5 集合均一致，**未观察到**题目列出的九类典型问题的表现

(Resize/CenterCrop 顺序、RGB/BGR 通道、插值方式、mean/std、Softmax 维度、eval 模式、BN 状态、导出方式、数值精度)；该结论只覆盖已测输入，**不等于「已排除九类问题」**。

范围限定：同口径 (n=12 真实图片) 的落盘产物只有三份——Pet-37 6.199e-06、

ImageNet M0.9 1.717e-05、ImageNet M1.0 1.812e-05 (见 report/LOGITS_AUDIT.md)；

其余型号只有导出与性能结果，**不做同口径一致性声明**。

13.4 结构重参数化在部署侧的体现

三个 ONNX 模型**全部是推理态** (图内 BatchNormalization = 0、Conv = 103)。导出脚本

deploy/export_onnx.py 的顺序**不可交换**：load_state_dict → eval() → fuse() → eval()；**训练态融合必错**

(BN 用的是 batch 统计量而非 running 统计量)。需要训练态对照图时用 --no-fuse 导出到非交付目录

(outputs/reparam/)，该图有 24 个 BatchNormalization、126 个 Conv。

十四、性能测试

14.1 测试协议（严格按题目第 28 页的统一复测建议）

项	值
推理后端 / 精度 / batch / 输入	ONNX Runtime 1.30.0 / FP32 / 1 / 224×224
实际 Execution Provider	CPUExecutionProvider (脚本打印实际值，不靠任务管理器)
预热 / 正式 / 线程	10 次 / 50 次 / intra_op = 4
CPU / 内存 / 操作系统	13th Gen Intel Core i7-13650HX / 15.8 GB / Windows 11 10.0.26200

14.2 结果

模型	mean (ms)	P50 (ms)	P95 (ms)	min	max	文件大小
repvit_m0_9_in1k	7.45	7.37	8.00	7.08	9.11	20.36 MB
repvit_m1_0_in1k	9.20	9.15	9.76	8.75	10.46	27.33 MB
repvit_m1_1_in1k	10.41	10.23	11.24	9.90	12.70	33.06 MB
repvit_m1_5_in1k	17.78	17.65	18.93	17.09	19.42	56.35 MB
repvit_m2_3_in1k	32.65	32.69	33.32	31.69	33.49	91.90 MB

repvit_m0_9_pet37	7.20	7.12	7.69	6.73	8.49	18.89 MB
-------------------	------	------	------	------	------	----------

14.3 如何解读 mean / P50 / P95

mean 容易被少数慢样本拉高；P50（中位数）代表「典型一次推理」的耗时，**最适合横向比较**；P95 反映尾延迟、直接决定最差体验。本机 P95/P50 ≈ 1.09 ，延迟分布很稳。

14.4 为什么更大的模型不一定按 MACs 比例变慢

M1.0 相对 M0.9：MACs +34.9%，P50 只 +24.1% (7.37 $\rightarrow$ 9.15 ms)；M1.5 相对 M1.1：MACs +69.9%，P50 +72.5%——都不是线性关系。反过来 repvit_m0_9_pet37 与 repvit_m0_9_in1k 的 MACs 几乎相同（只差分类头），P50 却相差 0.25 ms (7.12 vs 7.37)。原因：① **小模型受固定开销支配**（算子启动、内存分配、线程同步与小模型计算量同量级）；② **depthwise 卷积算术强度低**，每读一个权重只做很少的乘加，**瓶颈在内存带宽而非算力**；③ **分类头/输出维度会影响**——37 类的 Gemm 比 1000 类小得多，这部分差异被放大了。

14.5 集显部署（进阶任务）

本机不具备条件，结论与依据见附录 E。

十五、遇到的问题和解决方法

按题目要求，本节只列**影响实验结论**的关键问题；完整代码缺陷清单见附录 A。

15.1 官方 checkpoint 无法直接载入 timm 实现：直接加载得到 unexpected = 713 / missing = 599。排查发现官方是 features.N.* / classifier.classifier.* (713 键)，timm 是 stages.M.* / head.head.* (389 键)，**交集为 0**。解决：① 官方 ckpt 配 vendored 官方实现 (missing=2, unexpected=0)；② 实测证明 timm/HF 权重与官方 ckpt **逐位相同** ($\max|\Delta\text{logits}| = 0$)，迁移训练可直接从 timm 侧权重起步，省去手写 713 $\rightarrow$ 389 键转换器。

15.2 data/provided/ 是规格书虚构的路径：规格书要求读取考核方放在该目录的输入，但目录为空。把 29 页试题 PDF 全文提取后搜索，“provided”与“data/”各命中 0 次；PDF 第 28 页那段是「**建议考核方发布前准备内容**」，写给出题方，不是「已下发给候选人」。解决：改判为**结构性偏差**，Pet 划分与类别映射从官方归档自建、ImageNet 子集用官方 val 分层抽样并**显式声明是自建子集**。

15.3 训练预算不等价（最影响结论的问题）：opt_mix 在第 24 epoch 触发早停（共 25 epoch），baseline 跑满 40 epoch，两组预算差 37.5%，「训练更久」会混进增益，**控制变量实验不成立**。解决：early_stop_patience 由 8 改为 999，全部 8 组统一 40 epoch；并把预算等价性固化为可执行检查 (tools/same_budget.py 与 selfcheck 的 opt.budget)。

15.4 并发训练导致 DataLoader worker 被杀：同时跑多个训练进程时出现 `RuntimeError: DataLoader worker (pid(s) ...) exited unexpectedly`。本机 15.8 GB 内存，一个训练进程 + 4 个 worker 约占 3 GB，且崩溃的父进程会遗留 worker 僵尸进程（实测残留 8 个 python 进程占 8.2 GB）。解决：M07 全程**串行**执行、执行前清理残留进程（见 `_run_m07.sh` 注释）。

15.5 中文 Windows 的 GBK 编码：`json.load(open('outputs/env_snapshot.json'))` 抛 `UnicodeDecodeError`——验收命令用 `open()` 的**默认编码**（中文 Windows 上是 GBK），而该 JSON 含非 ASCII 字节（CPU 型号串含注册商标符），且仓库根目录路径本身含中文。解决：**全仓 JSON 落盘统一 `ensure_ascii=True`**（语义不变、文件变纯 ASCII，任何编码下都能读回），影响 30+ 个文件。

15.6 本机网络的多处不可达：

目标	状态	替代方案	保真度证明
github.com	HTTP 000	ghfast.top 前缀代理	字节数 + SHA256 与官方一致
www.robots.ox.ac.uk	超时	HF 镜像上的官方归档	MD5 与官方基准逐字符一致
commons.wikimedia.org	HTTP 000	ImageNet 跨集合真实照片	逐张登记来源与许可，并声明非原创

关键原则：绕过障碍不是问题，**绕过之后不证明保真度才是问题**；证明不了的，就如实标注 为「替代品」。

十六、总结与后续计划

16.1 主要结论

1. **RepViT 确实是纯卷积网络：**M0.9 在 1000 张自建 ImageNet 子集上 Top-1 = **78.20%**，与官方公布的 78.7% 仅差 0.5 个点。

2. **迁移到 Pet 37 类效果良好：**test Top-1 = **92.34%**、Macro-F1 = **92.22%**（唯一落盘

`outputs/metrics/baseline_test.json`: `top1 = 0.9234123739438539`、`macro_f1 = 0.9221970249315374`、`num_samples = 3669`、`eval_count = 1`），且几乎不混淆猫狗（跨物种错误仅占 **3.56%**，10 / 281）。

3. **结构重参数化：分支合并是代数等价的替换，实测误差小于阈值**——Pet-37 baseline 的 32 个固定随机输入上 $\max|\Delta\text{logits}| = 7.093\text{e-}06$ （阈值 $1\text{e-}4$ ）、Top-1 **32/32** 一致，推理态 ONNX 图的 BatchNormalization 节点从 24 降到 **0**、Conv 从 126 降到 **103**。该结论只覆盖已测输入与设定阈值，**不是位级完全相等**。

4. **部署闭环完整**：Pet-37 / ImageNet M0.9 / ImageNet M1.0 三个交付 ONNX 模型各 n=12 的 PyTorch-ONNX Top-1 与 Top-5 集合一致率均为 **100%**（落盘 `outputs/metrics/consistency_*.json`）；其余型号只有导出与性能结果，不做同口径一致性声明。三个交付模型的 P50 延迟为 **7.1 ~ 9.2 ms**（家族六个型号为 **7.1 ~ 32.7 ms**，见 14.2 节）。

5. **跨集合泛化**：8/8 判对只是小样本观察，不能证明跨域泛化或没有过拟合（8 张 ImageNet 跨集合实拍图，逐张登记来源与许可）。

6. **四项优化方法均未带来超出噪声的增益**（test Top-1 全部落在 91.93~92.53 的 0.6 个点窄带内，小于二项分布 95% 置信区间半宽 ± 0.9 个点）；组合消融显示 A 与 B 存在**超加性交互**（+0.54）。收敛速度上，三个优化臂的最优 epoch（A 31 / B 32 / A+B 31）都比 baseline（34）更早，§8.2「正则叠加会延长收敛」的预期**没有被证实**（单次训练、单种子、40 epoch 预算）。

16.2 工程与实验的可复现性

- 全部数字可在 `outputs/` 中溯源（每个 JSON 带 `command / timestamp / platform` 元字段）；
- **34 条 DoD 验收项**由 `tools/selfcheck.py` 逐条自检（实测 34 个 @check，报告见 `outputs/metrics/selfcheck_report.json`：summary = {total: 34, pass: 34, fail: 0}）；
- 无任何写死的个人绝对路径（paths 检查 **0 处**）：三个一致性 JSON 的 `onnx_path / images` 已规范为仓库相对路径（写盘代码 `deploy/compare_torch_onnx.py` 的 `repo_rel()`）；
- 控制变量由 `tools/diff_config.py` 强制校验，预算等价由 `tools/same_budget.py` 校验。

两项 logits 误差实验的复跑命令（写入 `outputs/verification/`，不覆盖正式产物）：

```
# 重参数化 (12.4 节)：Pet-37 baseline, 32 个固定随机输入 seed=20240912
python tools/reparam_verify.py --model repvit_m0_9_pet37 --weights checkpoints/baseline_best.pt \
    --num-samples 32 --batch-size 8 --seed 20240912 --skip-onnx --out-dir
outputs/verification/reparam_pet37

# PyTorch→ONNX (13.3 节)：pet_test.txt 按顺序前 12 张真实图片
python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 --images datasets/lists/pet_test.txt \
    --limit 12 --out outputs/verification/consistency_repvit_m0_9_pet37_n12.json
```

16.3 后续计划

INT8 动态量化 / 知识蒸馏（M2.3 蒸馏 M0.9） / 结构化剪枝；OpenVINO 与 DirectML 后端对比（需具备 集显 的机器）；置信度校准（温度缩放降低 ECE）；七类扰动鲁棒性测试（模糊、亮度、JPEG、遮挡、旋转、噪声、背景替换）；用 450e 权重作为起点并尝试 Layer-wise LR Decay。

附录 A：实测发现并修复的代码缺陷

全部由「按验收命令实跑 → 报错 → 定位到与假设不符处」暴露，**没有一项靠阅读代码发现**。

按类归并如下（完整逐条记录见 PROVENANCE.md）：

缺陷	影响	处置
parse_split.py: next(f) 前用 sum(1 for _ in f) 耗尽迭代器	StopIteration, 脚本不可用	改用一次性读入 + 计数
assert_data.py: 未排除 macOS .* 资源叉; ids() 未跳过 # 注释行	trimaps 计数翻倍 (7390→14780) ; 误报划分不一致	过滤 .* 与 # 行
visualize.py: 用 class_idx < 12 判猫; 跨物种错误率分母把 cross 重复计一次	物种判定整体错位; 指标恒等于 50%	改用官方猫狗交错映射; 修正分 母
reparam_verify.py: zip(...to list) 后再次 .tolist(); registry key 未解析	AttributeError; Unknown model, DoD #29/#30 不通过	修正链式调用; 接入 model_registry
train.py: load_config 不解析 _base_ 继承	全部 opt_*.yaml 运行时 KeyError	支持 _base_ 递归合并
compare_torch_onnx.py: 未 rsplit(None,1) 解析 list 行	整行当路径 → OSError: [Errno 22]	按空白右分割取路径段
selfcheck.py: weights_only=True 读自训 ckpt; deepcopy(m).fuse(); 盘符正则误 报 https://; 混淆矩阵 CSV 首列 是类名; opt.diff 只比顶层字典	5 项检查恒 FAIL 或误判单变量	逐项修正 (详见 PROVENANCE.md)
全仓 30+ 处 json.dump(..., ensure_ascii=False)	含中文的绝对路径让 open() 抛 UnicodeDecodeError	统一 ensure_ascii=True
draw_arch.py: 探针写死 stem.0; 形状链未合并重复值	结构图生成失败; 输出 224→112→56→56→28...	适配 timm 1.0.29 实际结构并合并 相邻重复

env_check.py: 把「不在 venv 中」判为致命错误	与「使用全局解释器」决策冲突, DoD #1 无谓 FAIL	降级为 WARN 并注明依据
----------------------------------	--------------------------------	----------------

共性结论：凡是能实测的都要实测；文档（哪怕是规格书）给的数字与代码都只是量级参考。

附录 B：产物索引

类别	路径
环境 / 参数 / 权重	outputs/env_snapshot.json; outputs/metrics/{count_params,count_flops,weight_sha256,weight_load_report}.json
数据审计	outputs/metrics/{leakage_check,dataset_report,pet_label_audit,split_audit}.json; datasets/lists/*
官方评价	outputs/pretrained_eval/<model>/{metrics,latency,top5_samples}.json、 predictions.csv、cases/
训练与测试	outputs/logs/<exp>_metrics.{csv,jsonl}、outputs/metrics/<exp>_test.json、 outputs/predictions/*
可视化	outputs/{curves,confusion_matrix,gradcam,predictions}/*、 outputs/advanced/interp/*
重参数化 / ONNX	outputs/reparam/*、onnx/<registry_key>.onnx、outputs/advanced/repvit_m0_9_*
性能 / 结构图	outputs/benchmarks/*、outputs/metrics/bench.jsonl、outputs/architecture/*
报告素材 / 自检	outputs/report_assets/*、outputs/metrics/selfcheck_report.json、 outputs/verification/*

附录 C：鲁棒性测试（7 类扰动 × 6 级 severity）

模型 repvit_m0_9_pet37, Pet test 抽样 150 张, CPU。以 JPEG q=75 为基线归一化的 MCE（越低越鲁棒）：
 旋转 **0.583**（最鲁棒） < 亮度 0.820 < 运动模糊 0.847 < JPEG 0.986（基线） < 遮挡 1.347 < 高斯模糊 1.431 < 高斯噪声 **1.444**（最脆弱）。高斯噪声 σ 从 0 升到 0.12 时, Top-1 从 94.0% 单调降到 78.0%，置信度从 0.781 降到 0.643，ECE 只从 0.167 升到 0.178（**置信度下降得比精度高**，说明模型在噪声下「知道自己不确定」）。

结论：对**几何变换（旋转）**最鲁棒、对**高频退化（噪声、模糊）**最脆弱，与 RepViT 全部使用 depthwise 卷积、感受野有限的结构特性一致——旋转不改变局部纹理统计，而噪声直接破坏

深度可分离卷积依赖的高频细节；遮挡维度（1.347）比预期脆弱，与 §11.4 的「部分样本依赖背景」互相印证。数据：outputs/advanced/robustness_repvit_m0_9_pet37.json。

附录 D：深入可解释性

outputs/advanced/interp/ 下的四类产物：**阶段特征图**（5 个挂载点：stem 56×56/48ch、stages.0 56×56/48ch、stages.1 28×28/96ch、stages.2 14×14/192ch、stages.3 7×7/384ch，形状由 forward hook 现测）；**多层 Grad-CAM**（同图挂 5 层，浅层碎、深层聚焦）；**t-SNE**（37 类在 GAP 特征空间的分布，猫科与犬科形成两个大簇）；**置信度校准**（温度缩放拟合值 $T = 0.615 < 1$ ，说明模型欠自信，与 §11.6 一致）。温度缩放只调 1 个参数、不改变预测类别，且只在验证集上拟合（在 test 上拟合属于测试集泄漏）。

附录 E：集显部署（进阶④）—— 本机不具备条件

逐项核查后结论：**本机无法完成集显加速部署**。依据（全部实测）：

`onnxruntime.get_available_providers()` = ['AzureExecutionProvider', 'CPUExecutionProvider']; WMI 列出的显示适配器**只有 NVIDIA GeForce RTX 4060 Laptop**，无 Intel/AMD 集显；OpenVINO EP 与 DirectML provider 均不存在 → ORT 警告并**回退 CPU**（脚本检测到回退后把该行标为 invalid）；OpenVINO 原生导入报 `ModuleNotFoundError: No module named 'openvino'`。

`tools/bench_igpu.py` 的设计满足题目「**不得只根据任务管理器 GPU 占用判断部署成功**」：它打印**实际生效的 Execution Provider**，并在检测到回退时标 invalid，而不是给一个看起来更快的假数字。产物：

outputs/benchmarks/igpu_{cpu,ort_ov,ov_native,dml}.json（其中 ort_ov / dml 的行为 invalid，记录的是回退后的 CPU 数值，**不得引用为集显性能**）。